

Prompt Engineering

Mehmet Ali Akyol, PhD

Ankara Medipol University

October 31, 2025

Week 4: Prompt Patterns & Frameworks

Where We Are

- Week 1: Prompt fundamentals and the power of structured instructions.
- Week 2: The mechanics of modern language models.
- Week 3: Principles and anti-patterns for effective prompt design.
- **This week:** Codifying best practice with reusable prompt patterns and frameworks.

Weekly Learning Objectives

- Differentiate between atomic prompt patterns and broader frameworks.
- Combine patterns to address complex, multi-step tasks.
- Select frameworks based on task type, risk level, and time available.
- Translate frameworks into templates that teammates can reuse.
- Evaluate tradeoffs between prescriptive templates and flexible prompting.

Key Concepts

- Pattern vs. framework vs. playbook
- Pattern stacks (role + format + reasoning)
- Prompt lifecycle frameworks (PLAN, DRAFT, REVIEW)
- Task diagnostics (clarity, constraints, context, checkpoints)
- Governance patterns (guardrails, refusal handling)

Atomic Patterns Recap

- **Role:** Persona, audience, tone setters.
- **Format:** Tables, JSON, bullet, code block.
- **Reasoning:** Step-by-step, chain-of-thought, critique.
- **Context:** Retrieved excerpts, facts, style guides.
- **Constraints:** Word limits, banned topics, sourcing rules.

Pattern Stacking

- Combine patterns intentionally: role + format + reasoning → reliable outputs.
- Use delimiters to isolate instructions, data, and examples.
- Add validation requests: “List assumptions before final answer.” or “Return JSON conforming to schema.”
- Introduce fallback logic: “If information missing, respond with: insufficient context.”

Adaptation Patterns

- **Surface shift:** Change tone or reading level without altering content.
- **Format shift:** Convert text to tables, checklists, timelines.
- **Perspective shift:** Rewrite from stakeholder viewpoints.
- **Language shift:** Translate while preserving style constraints.

- **CO-STAR**: Context, Objective, Style, Tone, Audience, Response.
- **RACI Prompt Canvas**: Role, Action, Constraints, Input.
- **IDEA Loop**: Identify task, Draft prompt, Evaluate output, Adjust.
- **SMART Prompt**: Specific, Measurable, Achievable, Relevant, Time-bound outputs.

Choosing a Framework

- **Low stakes:** Lightweight templates (role + format) suffice.
- **Moderate stakes:** Add evaluation and guardrails (IDEA Loop, SMART Prompt).
- **High stakes:** Multi-stage prompts with critique, verification, and escalation rules.
- Consider team adoption: Can colleagues follow it quickly?

Framework to Template

- Start with fields the requester fills (objective, audience, constraints).
- Encode defaults for tone, format, delimiters to ensure consistency.
- Provide example completions highlighting best practice.
- Offer optional add-ons (e.g., risk checks, summary level).

Deep Dive: CO-STAR Framework

- **Context:** *Summarize the following user feedback for a product manager.*
- **Objective:** *Extract the core feature request and the user's primary pain point.*
- **Style:** *Use clear, professional language.*
- **Tone:** *Neutral and objective.*
- **Audience:** *A busy product manager who needs the key takeaway in seconds.*
- **Response:** *Format as a JSON object with keys "feature_request" and "pain_point".*

Origin

Adapted from the classic RACI project management matrix (Responsible, Accountable, Consulted, Informed), which has been used for decades to clarify roles in complex projects.

- **Role:** *You are a helpful, expert-level programming assistant.*
- **Action:** *Refactor the provided Python code to be more idiomatic and efficient.*
- **Constraints:** *Use list comprehensions where appropriate and ensure the code adheres to PEP 8 standards.*
- **Input:** *The code to be refactored is provided below in the text block.*

Deep Dive: The IDEA Loop

Origin

A modern variant of classic continuous improvement cycles like PDCA (Plan-Do-Check-Act) or John Boyd's OODA Loop (Observe-Orient-Decide-Act), tailored for the iterative nature of prompt engineering.

- **Identify:** The goal is to get a concise, 3-bullet summary of a long report.
- **Draft:** The first prompt is simple: "Summarize this report."
- **Evaluate:** The output is a long paragraph, not the desired bulleted list.
- **Adjust:** The prompt is refined: "Summarize this report in exactly three bullet points."

Origin

Directly borrows from the widely-used SMART criteria for effective goal-setting, first appearing in a 1981 article by George T. Doran for defining project objectives.

- **Specific:** *Generate 5 multiple-choice questions about photosynthesis.*
- **Measurable:** *Each question must have exactly 4 options: 1 correct and 3 incorrect.*
- **Achievable:** *The questions should be suitable for a 9th-grade biology class.*
- **Relevant:** *The questions must cover the topics of chlorophyll and cellular respiration.*
- **Time-bound:** *This is a single-turn request; generate all 5 questions now.*

Advanced Pattern: Meta-Prompting

Concept

Using a prompt to have an LLM generate, critique, or refine another prompt.

- **Generator Prompt:** "Create a prompt for a junior developer to document a Python function."
- **Critique Prompt:** "Review this prompt. Is it clear? Does it specify the output format? Suggest one improvement."
- **Refinement Prompt:** "Given the following critique, revise the original prompt to be more effective."
- **Use Case:** Automating the creation and maintenance of a high-quality prompt library.

Advanced Pattern: Tool Use & Function Calling

Concept

Empowering an LLM to use external tools by providing a manifest of available functions and their schemas.

- The model doesn't execute code; it generates a JSON object specifying the tool to call and the arguments to use.
- **Example Flow:**
 - 1 User asks: "What's the weather in Istanbul?"
 - 2 Model returns: `{"tool": "get_weather", "city": "Istanbul"}`
 - 3 Your code executes the function and passes the result back to the model.
 - 4 Model generates a natural language response based on the tool's output.

Advanced Pattern: Dynamic Few-Shot Selection

Concept

Instead of using static, hard-coded examples, programmatically select the most relevant examples from a library based on the user's query.

- **How it works:**

- ① Store a large library of high-quality prompt/completion pairs.
- ② When a user query comes in, use a similarity search (e.g., vector embeddings) to find the most similar examples in your library.
- ③ Dynamically insert these top K examples into the prompt before sending it to the LLM.

- **Benefit:** Provides highly relevant, in-context learning for the model on-the-fly.

From Frameworks to Playbooks

Concept

A playbook is an organizational asset that goes beyond a single prompt, detailing a complete, multi-step workflow for a complex business process.

- **A Playbook Contains:**

- The business goal and success metrics.
- A sequence of prompts (a prompt chain).
- Human-in-the-loop review and approval steps.
- Guardrails and escalation paths for failures.
- Examples of good and bad final outputs.

- **Goal:** To make AI-driven processes reliable, repeatable, and scalable across a team.

- Context: national AI policy update, 1-page summary.
- Framework: CO-STAR + critique loop.
- Output: Executive summary, key impacts, stakeholder risks, action items.
- Guardrail: require citation markers linked to provided context IDs.

Customer Support Escalation

- Objective: classify tickets and draft replies.
- Pattern stack: role + format + reasoning + guardrail.
- Steps: detect sentiment, map to policy, produce compliant response, list disclaimers.
- Evaluation: JSON report with confidence fields and suggested follow-up.

Research Synthesis Pattern

- Inputs: annotated bibliography, research question, stakeholder.
- Framework: PLAN (Purpose, Lens, Audience, Next steps).
- Output: 3-section memo with citations, gaps, recommended experiments.
- Reflection: require identification of uncertain claims for human review.

Key Takeaways

- Patterns are building blocks; frameworks package them for reuse.
- Match framework complexity to task stakes and resources.
- Templates accelerate teams when paired with clear guardrails.
- Iterate: test, critique, and refine before broad rollout.

Thank You!

Dr. Mehmet Ali Akyol

mehmetali.akyol@gmail.com

Questions?